

Cours 29 — Les algorithmes gloutons

Semaine 29 ■ Manuel Hachette ch. 13 (§ 13.3) ■ 2×1 h

Objectifs

- Reconnaître la stratégie gloutonne : le meilleur choix local, tout de suite
- La dérouler sur le rendu de monnaie et le sac à dos
- Savoir qu'un glouton peut être optimal... ou pas — et le prouver par contre-exemple

1. La stratégie gloutonne

Beaucoup de problèmes demandent la **meilleure** solution parmi une multitude de possibles (le moins de pièces, le chargement le plus précieux...). Essayer toutes les combinaisons explose vite (pour 30 objets : 2^{30} , un milliard de possibilités — semaine 4!).

L'algorithme **glouton** (*greedy*) renonce à tout explorer : à chaque étape, il prend **le meilleur choix immédiat**, sans jamais revenir en arrière. Rapide, simple... mais myope : le meilleur choix local ne mène pas toujours au meilleur résultat global.

2. Le rendu de monnaie

Rendre 27 avec des pièces $\{10, 5, 2, 1\}$ en un minimum de pièces. Le glouton : toujours la plus grosse pièce possible — $27 \rightarrow 10 + 10 + 5 + 2$: quatre pièces, et c'est optimal.

```
def rendu(somme, pieces):
    """Renvoie la liste des pièces rendues (glouton).

    Précondition : pieces triées par ordre décroissant.
    """
    resultat = []
    for p in pieces:
        while somme >= p:
            resultat.append(p)
            somme = somme - p
    return resultat
```

Mais avec le système $\{4, 3, 1\}$ et la somme 6 : le glouton prend $4 + 1 + 1$ (trois pièces), alors que $3 + 3$ en fait deux. **Un contre-exemple suffit** à prouver qu'un glouton n'est pas optimal — le réflexe scientifique de la semaine 13 (le test rouge). Le système de l'euro est dit *canonique* : le glouton y est toujours optimal.

3. Le sac à dos

Un sac supporte 10 kg; des objets ont chacun une masse et une valeur; maximiser la valeur emportée. Trois gloutons candidats — par valeur décroissante, par masse croissante, ou par **rapport valeur/masse** décroissant :

```
def sac_glouton(objets, capacite):
    """Renvoie la liste d'objets choisie (glouton par rapport
    valeur/masse décroissant). Non garanti optimal !"""
    def rapport(o):
        return o["valeur"] / o["masse"]
    choix = []
```

```
reste = capacite
for o in sorted(objets, key=rapport, reverse=True):
    if o["masse"] <= reste:
        choix.append(o)
        reste = reste - o["masse"]
return choix
```

Le rapport valeur/masse est le plus malin des trois — et pourtant aucun n'est optimal en général (contre-exemples en exercices). Trouver l'optimum exact du sac à dos est un problème **difficile** au sens fort : aucun algorithme efficace connu (lien avec ch. 1.7 : les limites du calcul).

4. Quand le glouton brille

Il est **optimal** sur certains problèmes (monnaie canonique, et de célèbres problèmes de planification), et ailleurs il donne vite une solution **honorable** — souvent tout ce dont on a besoin. Le compromis exactitude/rapidité : dernier avatar du fil rouge correction-efficacité de l'année.

À retenir

- Glouton : le meilleur choix **local** à chaque étape, sans retour en arrière.
- Rendu de monnaie : optimal pour l'euro, pas pour tout système ($\{4, 3, 1\}$ et 6!).
- Sac à dos : glouton par valeur/masse — bon, pas garanti optimal; l'optimum exact est hors de portée efficace.
- Un contre-exemple suffit à réfuter l'optimalité.